CANChat Agent — User Manual

CANChat Agent — Complete User Manual

Publication-quality user guide, generated from and verified against the extension's source code. Where the code and older documentation disagree, the code wins and the difference is flagged.

Product: CANChat Agent (Chromium browser extension, Manifest V3) **Version documented:** 0.1.0 **Audience:** First-time users with little or no AI experience, plus administrators and reviewers.

Screenshots in this manual are captured automatically by the test harness (`tests/e2e/walkthrough.spec.ts` and `tests/e2e/manual.spec.ts`) against a deterministic offline mock model, so they show the real interface without exposing any private data.

Table of contents

1. Introduction
2. Installation
3. Quick start
4. Core concepts
5. Feature reference
6. Settings reference
7. Common workflows
8. Privacy and security
9. Accessibility review
10. Troubleshooting
11. FAQ
12. Quick reference (cheat sheet)
13. Known limitations

- Appendix A: Complete feature inventory
- Appendix B: Complete tool catalogue
- Appendix C: Implementation vs. documentation discrepancies

1. Introduction

What it is

CANChat Agent puts an **AI assistant in a side panel** next to your web pages. What makes it different from a normal chatbot is that **the browser is its toolset**: instead of only talking, it can open tabs, run web searches, read the pages you have open (including pages you are already logged in to), fill in forms, click buttons, collect data into tables, and write up the results — doing the things *you* would do, while asking your permission before anything that changes a page or sends data.

What problems it solves

- **"Summarize / explain this page"** without copy-pasting into another tool.
- **"Compare what's across my open tabs"** — research spread over many pages.
- **Searching sites you're logged in to** (an internal wiki, Jira, SharePoint) using your existing browser session — no separate sign-in.
- **Repetitive browser chores** — pulling a list of items into a spreadsheet, drafting a document, triaging tickets — turned into one request.
- **Keeping a private, on-device knowledge base** of pages you can ask about later.

Key capabilities (at a glance)

- Chat in a side panel, in **English or French**.
- **Bring-your-own model:** any OpenAI-compatible endpoint (OpenAI, Azure OpenAI, a local model such as Ollama or LM Studio, or a corporate gateway).
- **Browser automation** with a safety gate: every page-changing action asks for approval first.
- **Skills** (reusable instructions) and **App playbooks** (site-specific know-how).
- **Knowledge bases** — save pages on your device and ask questions across them.
- **MCP and WebMCP** tool integrations.
- **Document handling:** read PDFs, Word/PowerPoint/Excel files, and video captions; produce downloadable CSV/JSON tables and Word documents.
- **Voice prompts**, **page snapshots** for image-aware models, conversation **history with labels and search**, and one-file **backup/restore**.

Intended audience

Anyone who browses the web and wants help reading, researching, or acting on pages. No programming knowledge is needed. You do need access to one AI model endpoint (see Quick start); if you don't have one, ask whoever manages AI access in your organization for an endpoint URL, an API key, and a model name.

2. Installation

Browser requirements

- A **Chromium-based browser** — Google Chrome or Microsoft Edge — **version 116 or newer** (the extension uses the Side Panel API and Manifest V3).
- Permission to load an extension. CANChat Agent is distributed as an *unpacked* extension (a folder), not from the Chrome Web Store.

What it can access, and why

When you load it, the browser will note that the extension requests these capabilities (declared in `public/manifest.json`):

Capability	Why it's needed
Side panel	To show the assistant beside your pages.
Tabs / active tab	To see your open tabs and read the one you're looking at.
Scripting	To read and (with your approval) operate page content.
Storage / unlimited storage	To save your settings, skills, memory, and knowledge bases **on your device**.
Search	To run queries through your default search engine.
Bookmarks	So you can reference a saved bookmark by typing `@`.
Tab groups	To gather the tabs it opens into one named group per conversation.
Offscreen	To build the on-device knowledge-base store and generate documents.
Downloads	To save files you ask for (tables, Word docs, exported chats, backups) — always via a **Save As** dialog so you pick the name and folder.
Access to all websites (`<all_urls>`)	So it can read and act on whatever site you ask about.

> **Plain-language note:** "Access to all websites" sounds broad, but the agent only does something on a page when *you* ask, and it must get your approval before changing anything. See Privacy and security.

Installation steps (load unpacked)

1. **Build the extension** (one-time, requires Node.js):

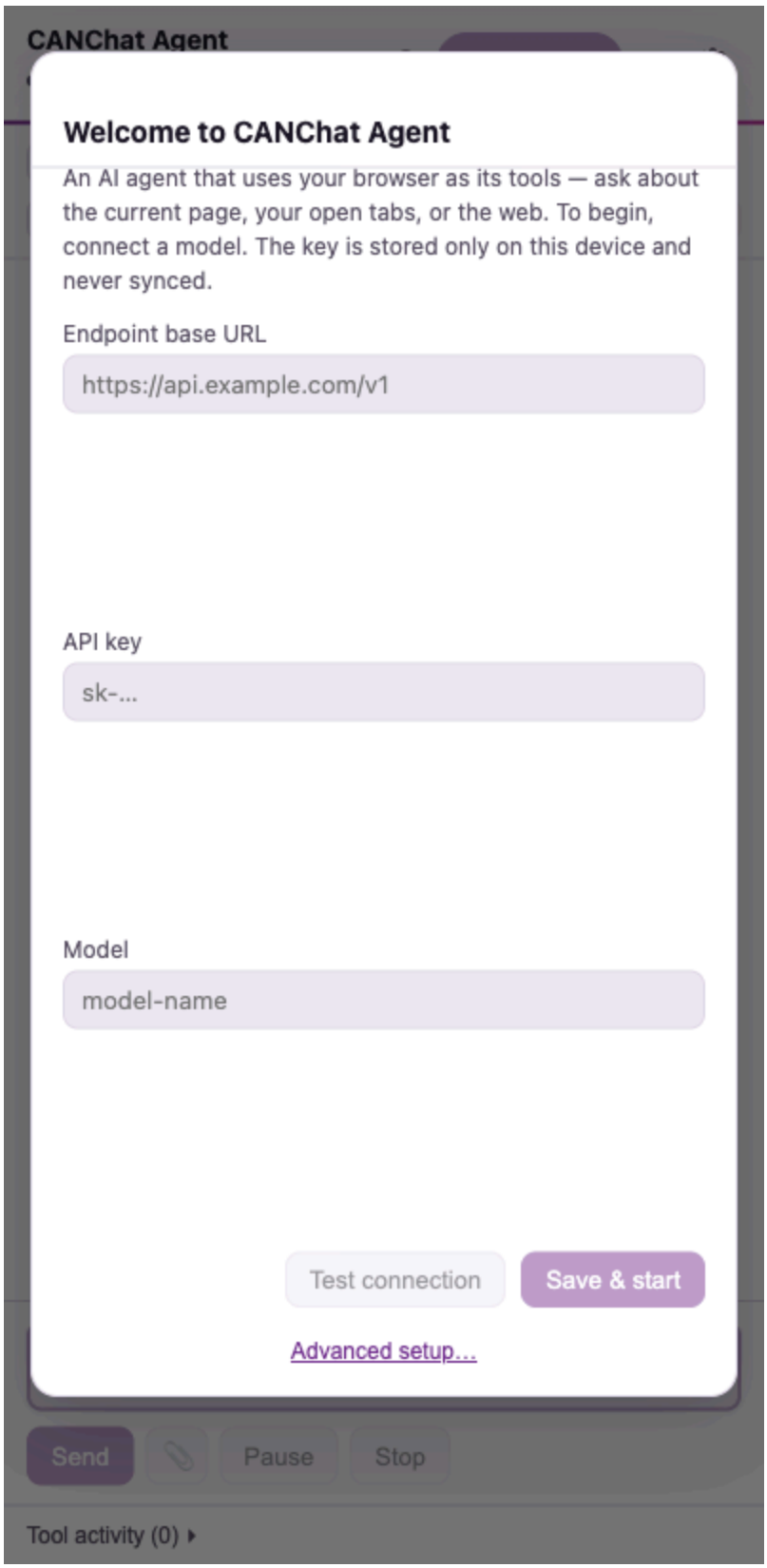
```
npm install
npm run build
```

This produces a `dist/` folder — the loadable extension.

2. Open your browser and go to `chrome://extensions` (Chrome) or `edge://extensions` (Edge).
3. Turn on **Developer mode** (top-right toggle).
4. Click **Load unpacked** and select the `dist/` folder.
5. The **CANChat Agent** icon appears in your toolbar. Pin it for easy access.

First launch experience

Click the toolbar icon. The side panel opens on the right. Because nothing ships preconfigured, the **first run shows a short welcome** asking for just three things — endpoint, key, and model — rather than the full settings screen:

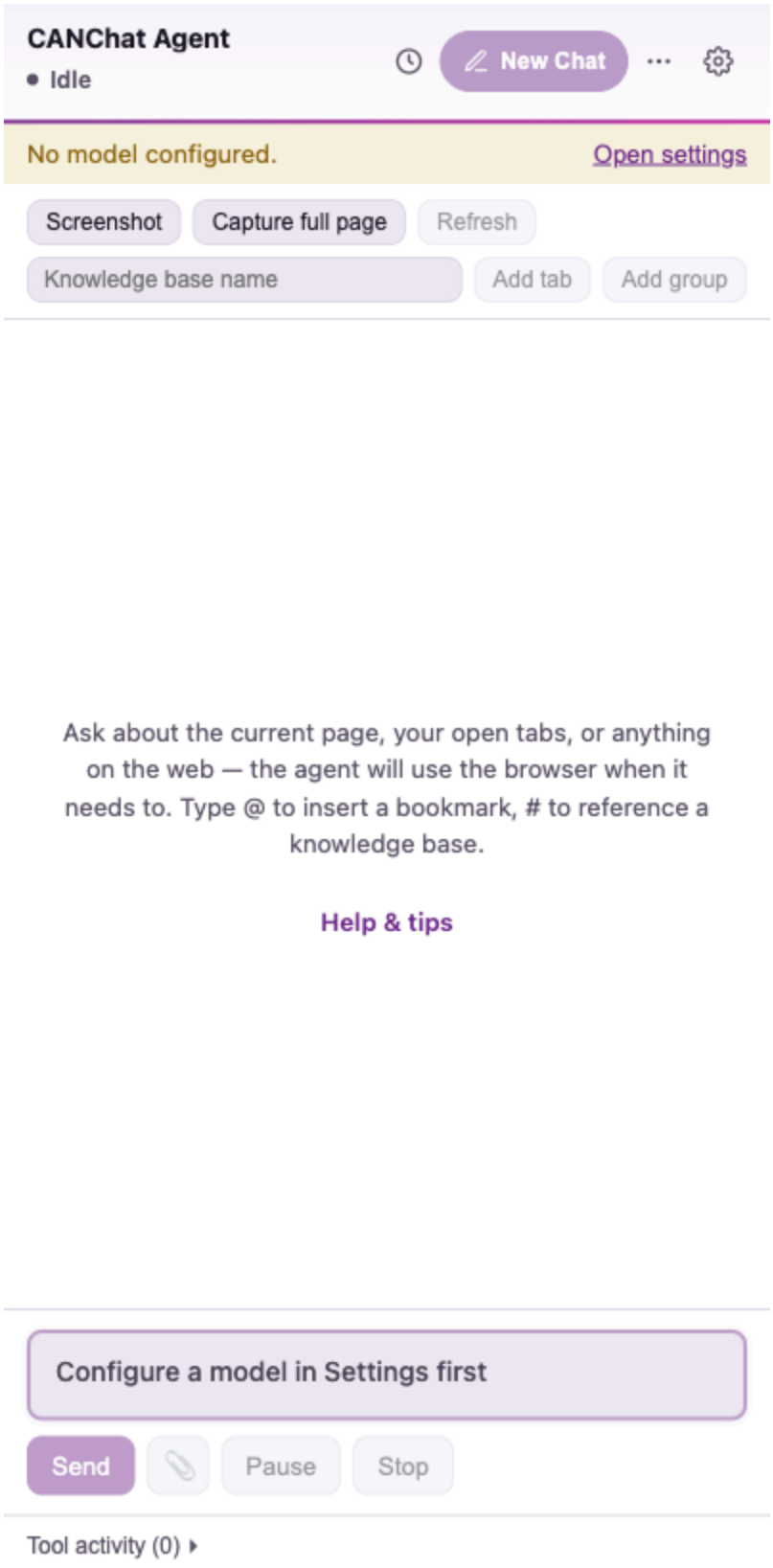


First-run welcome with three fields

What you're seeing: the **Get started** card with **Endpoint base URL**, **API key**, and **Model**, a **Test connection** button, a **Save & start** button, and an **Advanced setup…** link (bottom) that opens the Workspace **Models** page for Azure, embeddings, and other

options. Fill these in and you're ready; see Quick start.

If you close setup without saving a model, the panel shows a gentle reminder banner and the agent won't run until configured:



No model configured banner

---

3. Quick start

Step 1 — Connect an AI provider

In the welcome card (or **Workspace → Models**), fill in:

Field	What to enter	Examples
Endpoint base URL	The address of an OpenAI-compatible API	`https://api.openai.com/v1` · `http://localhost:11434/v1` (Ollama) · `http://localhost:1234/v1` (LM Studio) · your organization's gateway URL
API key	The secret key for that endpoint (hidden as dots)	`sk-…`
Model	The exact model name the endpoint expects	`gpt-4o`, `llama3.1`, or whatever your provider lists

> **Tip — what's a "model"?** The model is the specific AI brain you're talking to. Different endpoints offer different models; type the name your provider tells you. For browser automation, choose a model that supports **tool calling** (most modern chat models do). For reading screenshots, choose a **vision-capable** model.

Step 2 — Test and save

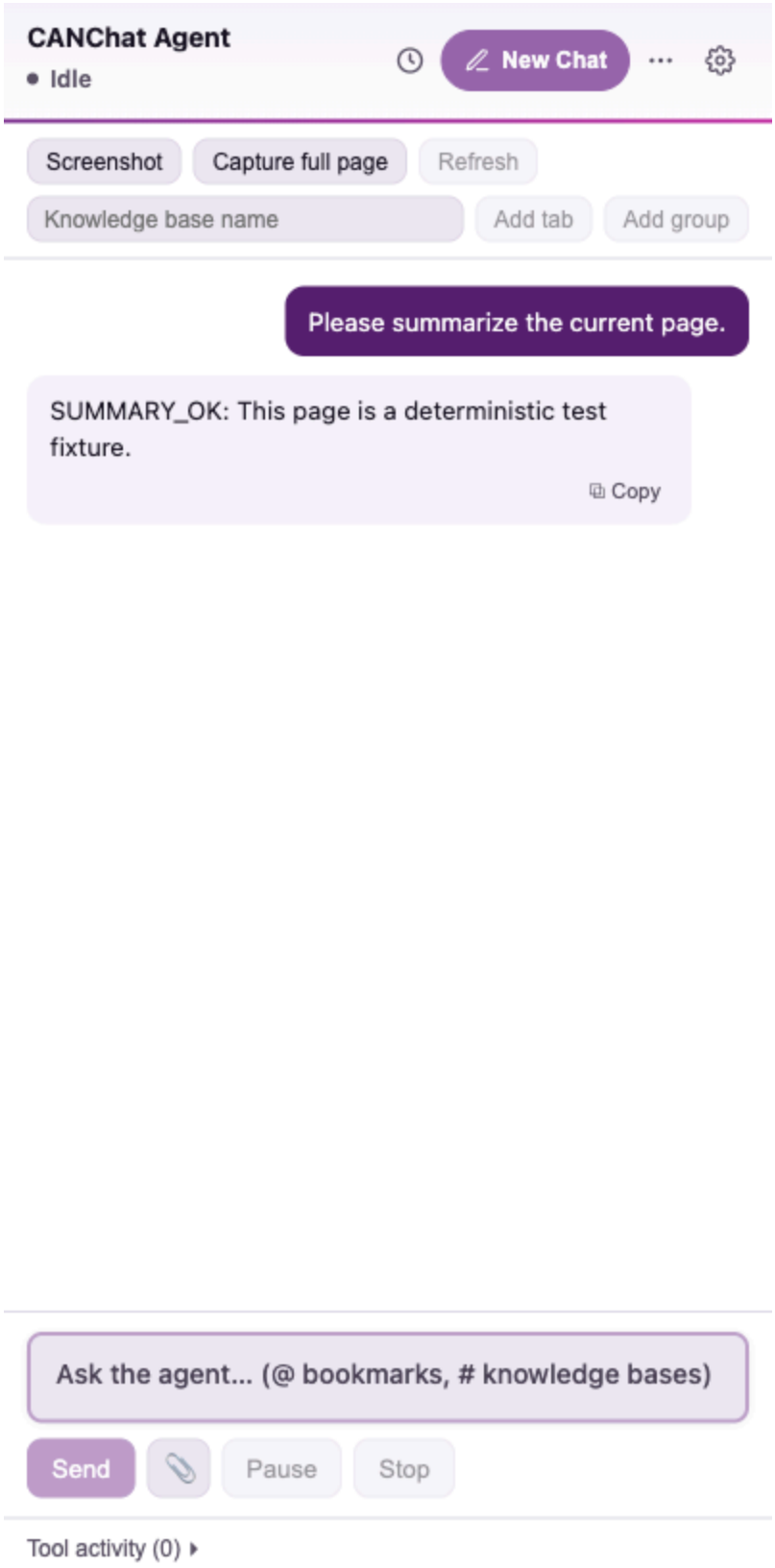
Click **Test connection**. The extension sends one tiny request and reports success or the exact error. When it succeeds, click **Save & start**.

Step 3 — Send your first request

Open any web page (say, a news article). In the side panel composer at the bottom, type a question and press **Enter** (use **Shift+Enter** for a new line):

> *"Summarize this page in five bullet points."*

The status pill shows **Thinking**, then **Acting** as it reads the page, and the answer appears as a chat bubble with a **Copy** button:



A chat answer in the side panel

What you're seeing: the header (product name + status), the **browser-tools toolbar** (Screenshot · Capture full page · Refresh, and the knowledge-base row), your message (purple bubble, right), and the assistant's answer (left) with **Copy**. The composer hint

reminds you that `@` inserts a bookmark and `#` references a knowledge base.

Step 4 — Use the browser tools

Try a request that needs the live web:

> *"Search the web for the latest on and give me three sources."*

The agent runs a search in a new tab, reads the results, and answers with a **Source tabs:** list of links. Tabs it opens are gathered into a **named tab group** for that conversation (e.g. *Loutre* or *Wolf*).

---

4. Core concepts

Short, plain definitions for the terms used throughout this manual.

Term	What it means here	Why it matters
Agent	The AI assistant that can *act* (use tools), not just chat.	It can do multi-step jobs: search, read, click, summarize.
Model	The specific AI you connect to via an endpoint.	You choose and pay for it; its abilities (tools, vision) set what's possible.
Endpoint / provider	The web address (and key) of your model service.	Nothing ships configured — you supply this once.
Tool	One concrete action the agent can take (read a tab, click, search…).	The full list is what the agent "can do" — see Appendix B.
Approval	A prompt asking you to allow a page-changing action.	Your safety gate — nothing is clicked, typed, or submitted without it.
Plan	The agent's step-by-step outline for a bigger task, shown to you.	Lets you watch progress and see what it intends to do.
Skill	A reusable set of instructions you save, invoked by `/name`.	Turns a repeatable task into a one-word command.
App playbook	A skill tied to a website that loads automatically there.	Teaches the agent how to drive a specific app (e.g. Gmail).
Learn mode	A recording mode that captures your clicks and form interactions on a site and	Lets you teach the agent by doing the task once.

Term	What it means here	Why it matters
	turns them into a reusable playbook.	
MCP	"Model Context Protocol" — an external tool server the agent can call.	Connects the agent to services beyond the browser.
WebMCP	Tools a *web page itself* offers to the agent (`navigator.modelContext`).	The agent uses the page's own actions instead of clicking around.
Knowledge base / repository	Pages you save **on your device**, searchable later.	Ask questions across saved pages ("RAG"); nothing is uploaded.
RAG	"Retrieval-augmented generation" — answering from your saved passages.	How knowledge bases produce cited answers.
Tab group	The named set of tabs the agent opens for one conversation.	Keeps research tidy; you can refer to it by name.
Context	The pages/tabs currently included for the agent to consider.	Controls what the agent "sees" by default.
Memory	Durable facts about you the agent may save (opt-in).	Personalizes answers; stored only on your device.
Lesson	A behavioral tip the agent saves from a past task (opt-in, shares the Memory toggle) — e.g. "use the Outlook tool before clicking around Outlook.com."	The agent gets better at *how* it works over time, without you teaching it explicitly.
Project	A named workspace that scopes conversations, memory, skills, and known sites.	Switch between client work, personal use, etc. without them mixing — nothing not assigned to a project is ever hidden.

5. Feature reference

5.1 The chat composer

Purpose: ask the agent anything. **How it works:** type and press **Enter** to send; **Shift+Enter** adds a line.

- **`@` — insert a bookmark.** Type `@` then part of a bookmark's name; pick from the menu to drop in that page's URL. The agent will open and read *that exact page*. The menu searches your **entire** bookmark folder tree (not just recent ones) plus your **Known sites**, so a Known Site is mentionable even if it isn't also a browser bookmark; matches on title, URL, folder, and description, with title matches ranked first.
- **`#` — reference a knowledge base.** Type `#` then a knowledge-base name; the agent answers from that saved collection.
- **`/` — run a skill.** Type `/` to see your skills (and the built-in `/learn`); pick one to apply its instructions.

Tip: mentions are explicit instructions — use them when you want the agent to use a *specific* page or knowledge base rather than guessing or web-searching.

5.2 Browser-tools toolbar (Context panel)

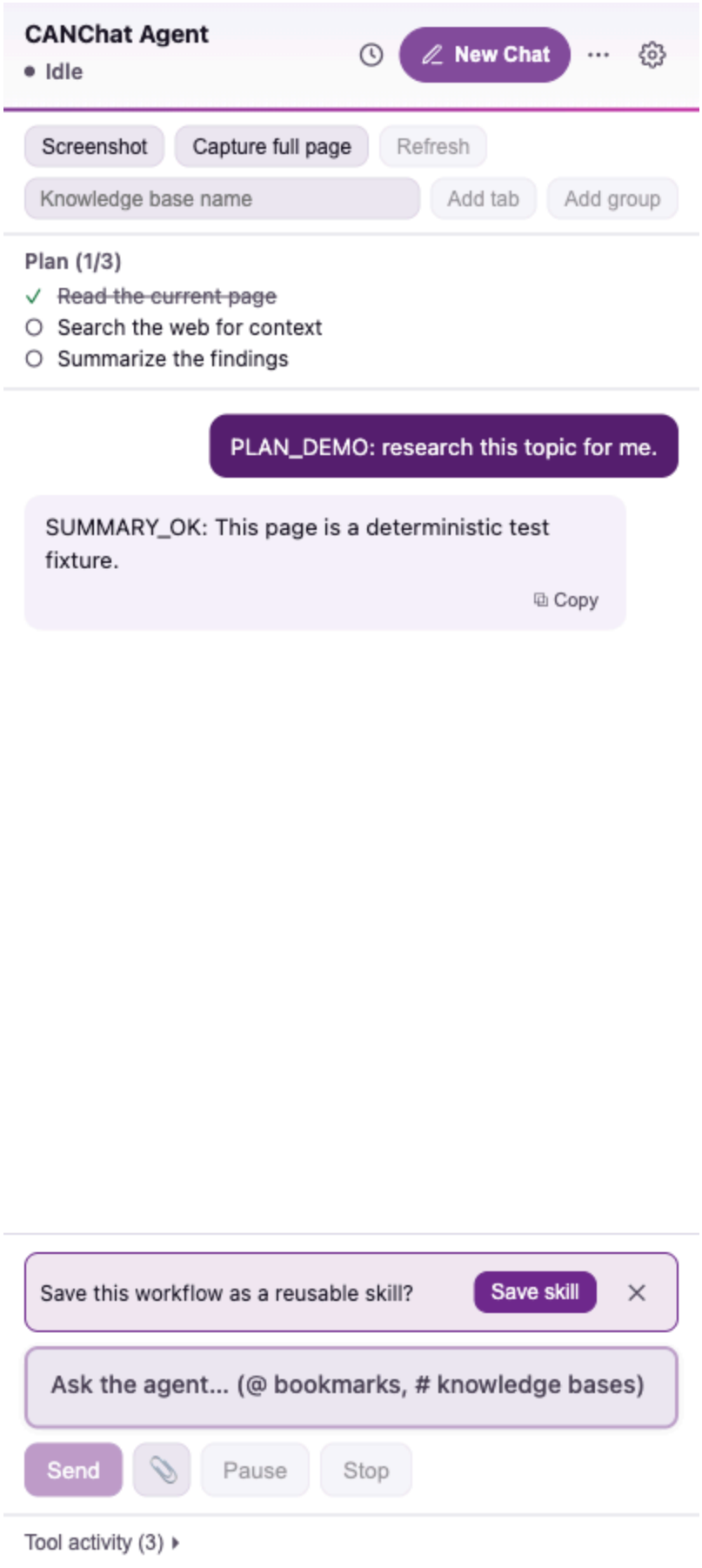
Just under the header sits the page-context toolbar (visible in the chat screenshot):

- **Screenshot** — capture the visible part of the current tab and attach it as an image for a vision-capable model to read. (Downscaled automatically; restricted pages like `chrome://` or the Web Store can't be captured.)
- **Capture full page** — add the whole current page to the conversation.
- **Refresh** — re-read the included tabs (enabled once context exists).
- **Knowledge-base row** — type a name, then **Add tab** (save the current page) or **Add group** (save every tab in this conversation's group) into that on-device knowledge base.
- **Skill buttons** — any skill you mark "show as a button" appears here for one-click launching.

The panel also lists included tabs with a **status dot** — green = read OK, amber = needs login, red = blocked/unsupported — and a **stale** tag if a page was read more than five minutes ago.

5.3 Plans and tool activity (watching the agent work)

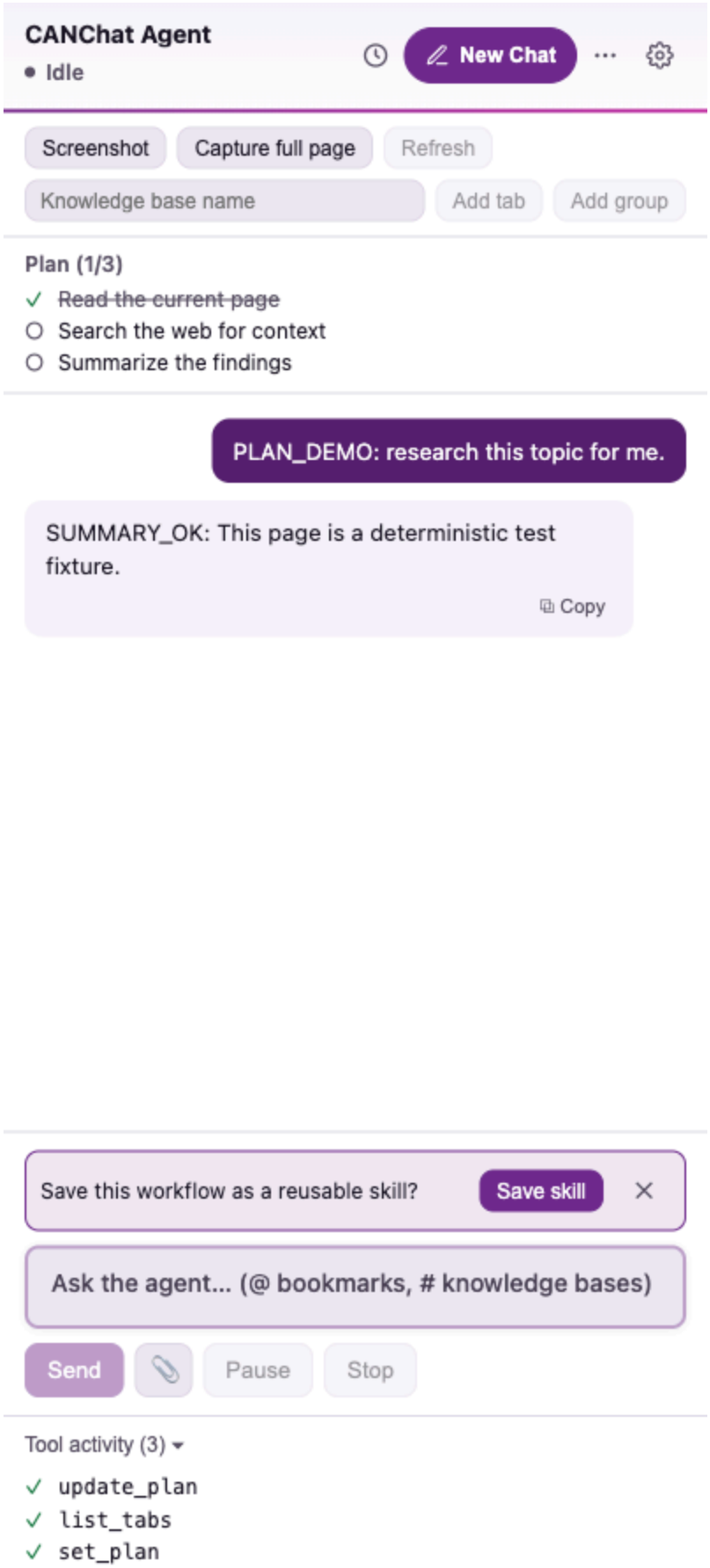
For multi-step jobs the agent first lays out a **plan**, shown above the chat, and logs each tool it runs in a collapsible **Tool activity** list at the bottom:


CANChat Agent
Idle
New Chat
Screenshot
Capture full page
Refresh
Knowledge base name
Add tab
Add group
Plan (1/3)
Read the current page
Search the web for context
Summarize the findings
PLAN_DEMO: research this topic for me.
SUMMARY_OK: This page is a deterministic test fixture.
Copy
Save this workflow as a reusable skill?
Save skill
Ask the agent... (@ bookmarks, # knowledge bases)
Send
Pause
Stop
Tool activity (3)

Plan panel and a finished answer

What you're seeing: **Plan (0/3)** with three steps (they tick off live as the agent works), your request, the final answer, the **Tool activity (2)** counter, and — because this was a substantial task — a **"Save this workflow as a reusable skill?"** offer (see Skills).

Expand the activity log to see exactly which tools ran:

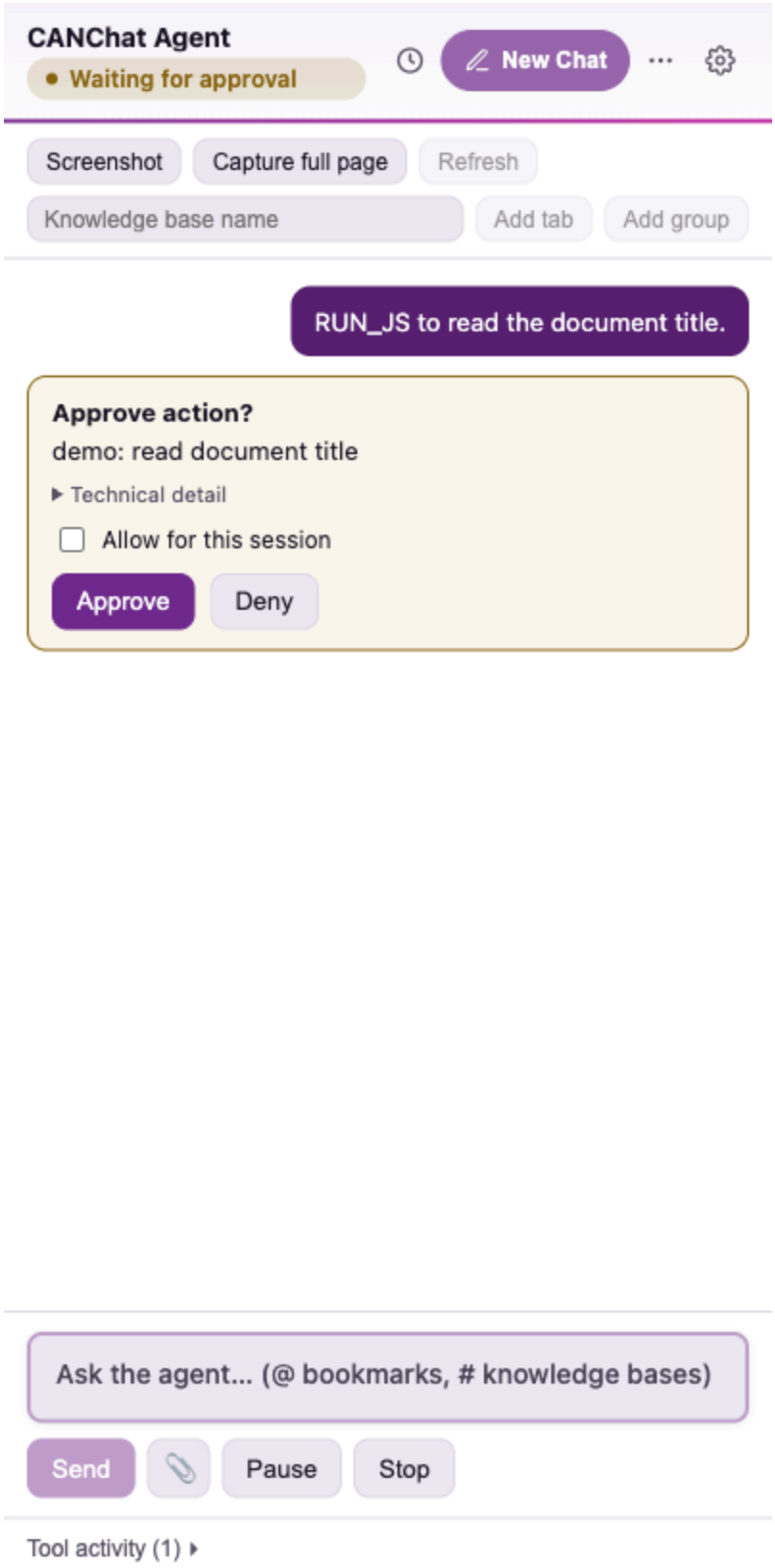

CANChat Agent
Idle
New Chat
Screenshot
Capture full page
Refresh
Knowledge base name
Add tab
Add group
Plan (1/3)
Read the current page
Search the web for context
Summarize the findings
PLAN_DEMO: research this topic for me.
SUMMARY_OK: This page is a deterministic test fixture.
Copy
Save this workflow as a reusable skill?
Save skill
Ask the agent... (@ bookmarks, # knowledge bases)
Send
Pause
Stop
Tool activity (3)
update_plan
list_tabs
set_plan

Tool activity log expanded

5.4 Approvals, logins, and permissions

The agent stops and asks before anything risky or blocked:

Approval — before it clicks, types, submits a form, runs JavaScript, reads all your tabs, or calls an external tool:



Approval prompt

What you're seeing: **Approve action?** with a plain-language reason, a **Technical detail** expander, and **Approve / Deny** buttons. Nothing happens until you choose.

Login required — if a page redirects to a sign-in wall, the task pauses with a card offering **Resume** (after you log in, in the normal browser) or **Stop task**.

Site permission — if the agent needs access to a site it hasn't been granted, a card offers **Allow this site**, **Allow all sites**, or **Stop task**.

> 📷 **Screenshot to capture manually — browser permission dialog.** After you click **Allow this site**, the *browser's own* permission dialog appears ("Allow CANChat Agent to read and change your data on ?"). *Why it isn't auto-captured:* this is native browser chrome, outside the extension's pages, so the test harness can't render it. *To capture:* trigger a task on a site the extension hasn't been granted, click **Allow this site**, and screenshot the browser prompt. *Suggested file:* `docs/user-guide/screenshots/06-permission-dialog.png`.

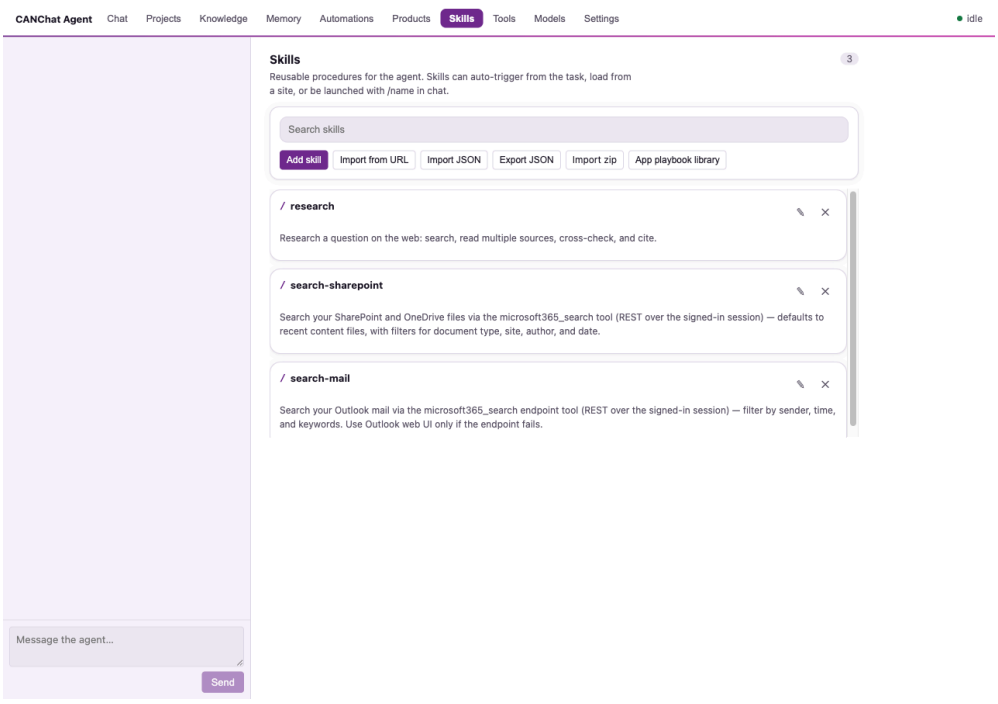
5.5 Status pill and run controls

The header status pill reads **Idle**, **Thinking**, **Acting**, **Paused**, **Awaiting approval**, **Auth required**, or **Error** (the dot gently pulses while working). Next to the composer:

- **Send** — submit your message.
- **Pause / Resume** — hold the agent between steps and continue later.
- **Stop** — end the current task immediately.

5.6 Skills

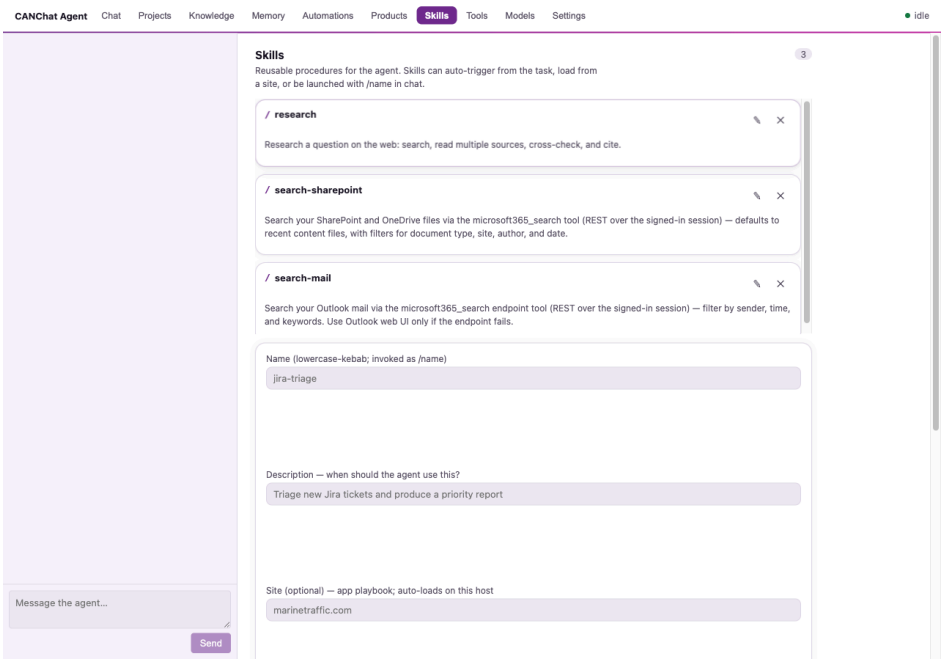
Purpose: save a procedure once, reuse it forever. **Where: Settings → Skills**.



Skills settings tab

What you're seeing: the **seeded skills** — **`/research`**, **`/search-sharepoint`**, **`/search-mail`**, and **`/map`** — each editable (✎) or removable (✕), and the toolbar: **Add skill**, **Import from URL**, **Import JSON**, **Export JSON**, **Import zip**, and **App playbook library**.

- **Run a skill:** type `/name` in the chat, or let the agent apply one automatically when your request matches its description.
- **Create one** with **Add skill**:



Skill editor form

  Fields: **Name** (lowercase-with-hyphens, becomes `/name`), **Description** (when the agent should use it), **Site** (optional — makes it an *app playbook* that auto-loads on that host), **Button label** + **Show as a button** (pin it to the toolbar), and **Instructions** (markdown steps).

- **Import** from a GitHub `SKILL.md` link, from pasted JSON, from a **zip** file (a single skill or a "pack" of several bundled together), or install a **curated App playbook** for Outlook on the web, Outlook.com, Gmail, MarineTraffic, or Jira Cloud. A `SKILL.md` can declare a `version:` in its frontmatter; re-installing the same skill only replaces it when the incoming version is equal or newer, so an older bundle can't overwrite a newer local edit and re-installing the identical file twice is a no-op.
- **`/learn` (built-in):** type `/learn` on a site and the agent explores it and saves an app playbook so it can operate that app next time.
- **Ask the agent to save a skill:** say "save this as a skill" (during or right after a task) and the agent turns the request into one itself — no new sandbox, no new code-execution ability; it's the same instruction-only skill format, and the task that produced it still ran through the normal approval-gated tool loop. Like any state-changing action, it asks you to approve before saving. Re-saving an already-saved skill bumps its version instead of creating a duplicate.

- **Auto-offer:** after a substantial task the agent also offers to **distill** the workflow into a new skill via a button (the chip in §5.3) — the same packaging step as above, just triggered from the UI instead of by asking.

5.7 Knowledge bases (save pages and ask later)

Purpose: keep a private, on-device library of pages and ask questions across them, with citations. **Add pages** with the toolbar's knowledge-base row (**Add tab / Add group**) or by asking the agent to "save this page to a knowledge base called X." **Ask** by typing `#name` in the chat, or "search my X knowledge base for …". **Manage** them under **Workspace → Knowledge** (see documents, chunk counts, and delete).

> Everything is stored **on your device** (in the browser's private file storage, OPFS). By default, text is turned into search vectors by an **on-device** model (no network) — see Embeddings to switch to an external endpoint, and Privacy.

Index a local folder. In **Workspace → Knowledge**, **drag a folder from Finder/Explorer onto the drop box**. Its files and subfolders (text, Markdown, PDF, Word/PowerPoint/Excel) are read, embedded **on your device**, and made searchable. Drag the same folder again later to re-index only what changed. (Folder indexing uses drag-and-drop on purpose — the OS "choose folder" dialog crashes on some Chrome/macOS builds.)

Index your Office 365 mailbox. This uses **Microsoft Graph** (Microsoft's official mail API) via a one-time sign-in — it needs a small **Azure AD app registration** first (your IT admin can set this up in a few minutes; see below), because it works reliably regardless of which Outlook web experience your organization uses. Once a **Client ID** is set in **Workspace → Models → Advanced**, go to **Knowledge** and click **Connect & index** — this opens a normal Microsoft sign-in/consent window once, then indexes your mailbox. Messages are embedded **on your device**. Re-run later (the button becomes **Index my Outlook mailbox**) to add only new messages. (A whole mailbox can take a while on the first pass.) **Disconnect** revokes the local connection at any time.

Setting up the Azure app (one-time, your IT admin may need to do this): register an app in Azure AD, add a redirect URI matching this extension's `chrome.identity` redirect (shown in the app's docs/repo), and request the delegated permissions **Mail.Read**, **Mail.ReadWrite** (needed even just to create a draft — there's no narrower scope for that), and **Calendars.Read**, plus **offline_access** and **openid**. Most organizations require an admin to grant consent for this combination. Paste the resulting **Client ID** into Workspace → Models → Advanced (Tenant is optional — leave blank unless your organization needs a specific tenant id).

Once you've indexed it at least once, you can turn on **Auto-refresh hourly** to keep the mailbox current without clicking Index again — it quietly checks for new mail in the background over your existing connection. It's off by default and never runs the first full index (or the first sign-in) on its own; if the connection expires, the next auto-refresh just records the failure (shown under the toggle) instead of interrupting you.

Searching a knowledge base is smarter than plain keyword or plain semantic search. When you ask a question against a knowledge base, the agent combines meaning-based search with exact keyword matching (so an exact ID, code, or name still surfaces even if

it's phrased differently), tries a couple of rephrasings of your question automatically, and does a final relevance pass over the top candidates before answering. This costs a little extra time per search in exchange for better answers; no setting to configure beyond the **Hybrid search** toggle in Workspace → Models → Advanced (on by default).

5.8 Documents in and out

- **Read PDFs** — including one open in the current tab ("summarize this PDF"). Scanned image-only PDFs have no text to extract.
- **Read Office files** — `.docx`, `.pptx`, `.xlsx` the browser downloaded instead of displaying. (Legacy `.doc/.xls/.ppt` are not supported.)
- **Read video captions** — "summarize this YouTube video" reads its transcript instantly (only if the video has captions).
- **Export a table** — for "collect X into a list" tasks, the agent emits a **data card** with **Download CSV**, **Download JSON**, and **Copy CSV**.
- **Create a Word document** — "write this up as a Word doc" produces a download card for a `.docx` file.

> **Saving files always asks where.** Every download — CSV/JSON tables, Word documents, exported conversations, and backups — opens a **Save As** dialog so you can rename the file or choose a folder, rather than it dropping silently into your Downloads folder.

5.9 Voice prompts

If you set a **transcription model** (Workspace → Models → Advanced), a **microphone** button appears beside Send. Tap to record, tap again to stop; your speech is transcribed into the composer. *One-time setup:* the side panel can't show the microphone permission prompt itself, so the first time it opens a small tab where you allow the mic; then return and tap again.

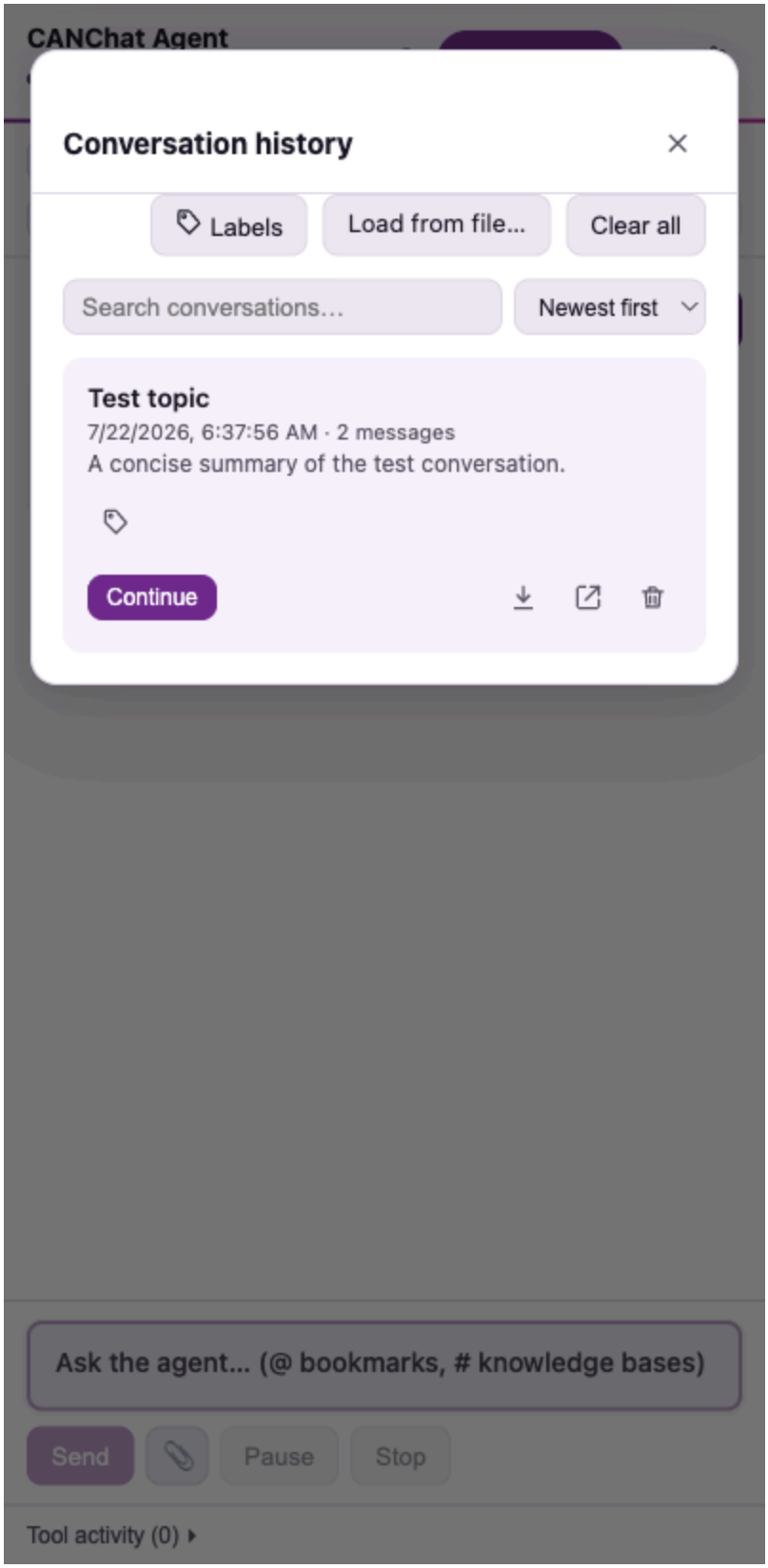
> 📷 **Screenshot to capture manually — microphone permission prompt.** The one-time grant tab (`microphone.html`) and the browser's native "Use your microphone?" prompt. *Why it isn't auto-captured:* the OS/browser microphone permission dialog is native chrome and requires real hardware access. *To capture:* set a transcription model, tap the mic in the side panel, and screenshot the grant tab and the browser prompt. *Suggested file:* `docs/user-guide/screenshots/07-mic-permission.png`.

5.10 Snapshots (for image-aware models)

The **Screenshot** button attaches a picture of the current tab. Use it for charts, maps, or canvas apps whose text the page tools can't read — a vision-capable model reads the image directly. Pending snapshots show as thumbnails above the composer with a discard (✕) option.

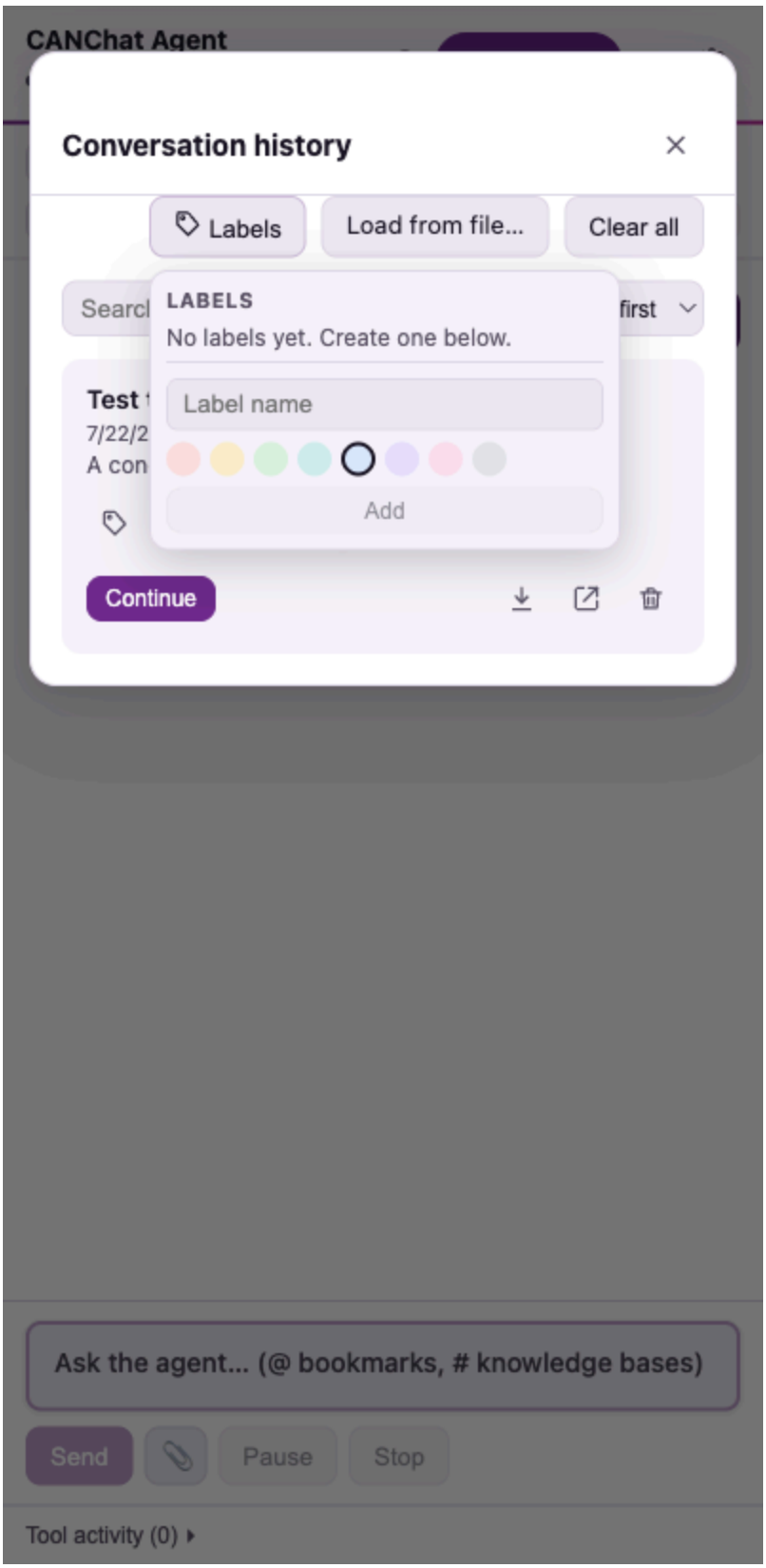
5.11 Conversation history, labels, and search

Open **History** (clock icon, top-right). Conversations are saved automatically.



History with search, sort, and labels

- **Search** conversations by title/preview; **sort** Newest or Oldest first.
- **Continue** reopens a conversation; per-row icons let you **export** it (download) and **delete** it; **Clear all** empties the list; **Load from file…** imports a previously exported conversation.
- **Labels** — create colored, named labels and filter the list by them:



Label picker

5.12 “New chat” and saving the current conversation

In the header: **New Chat** starts a fresh conversation (your current conversation is kept in History — it isn’t deleted). **Save conversation as HTML** lives in the ⋯ **More** menu.

5.13 The More menu, text size, and language

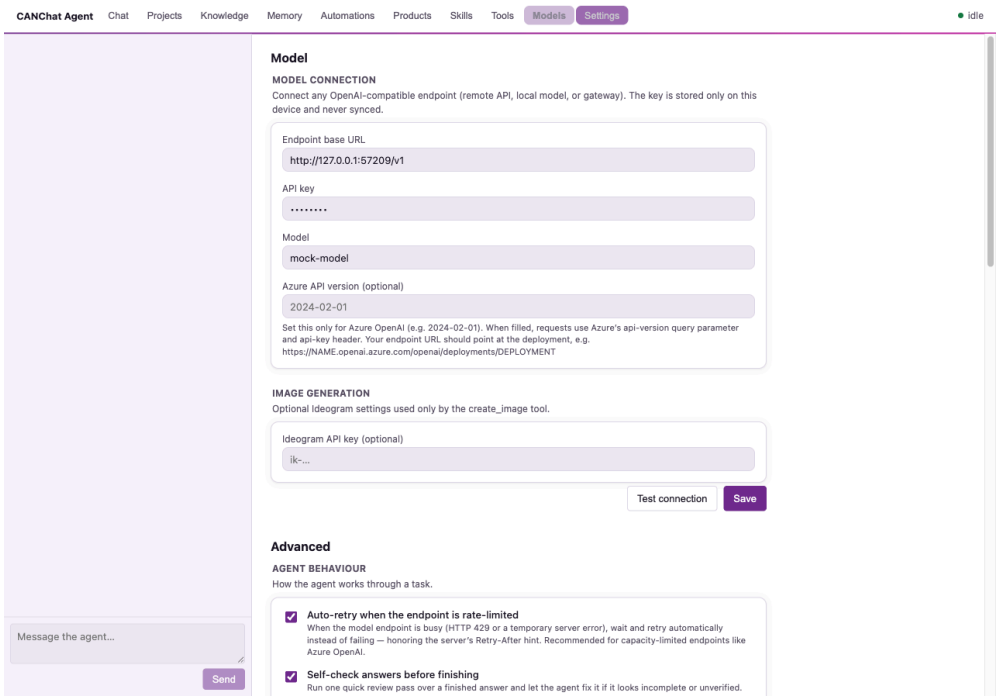
The header’s ⋯ **More** menu collects the less-frequent actions: the **`A−  100%  A+`** text-size control (scales the whole panel; click the percentage to reset), **Save conversation as HTML**, **Undo last exchange**, and **Learn mode**. Language (Auto / English / Français) is in **Workspace → Settings** and switches the whole interface immediately.

---

6. Settings reference

Click **Settings** (gear icon) in the side-panel header — it opens the **Workspace**, a full browser tab where all configuration lives. Model and agent options are on the **Models** page; skills, knowledge bases, tools, memory, and backup each have their own page (described below).

Models page — connection



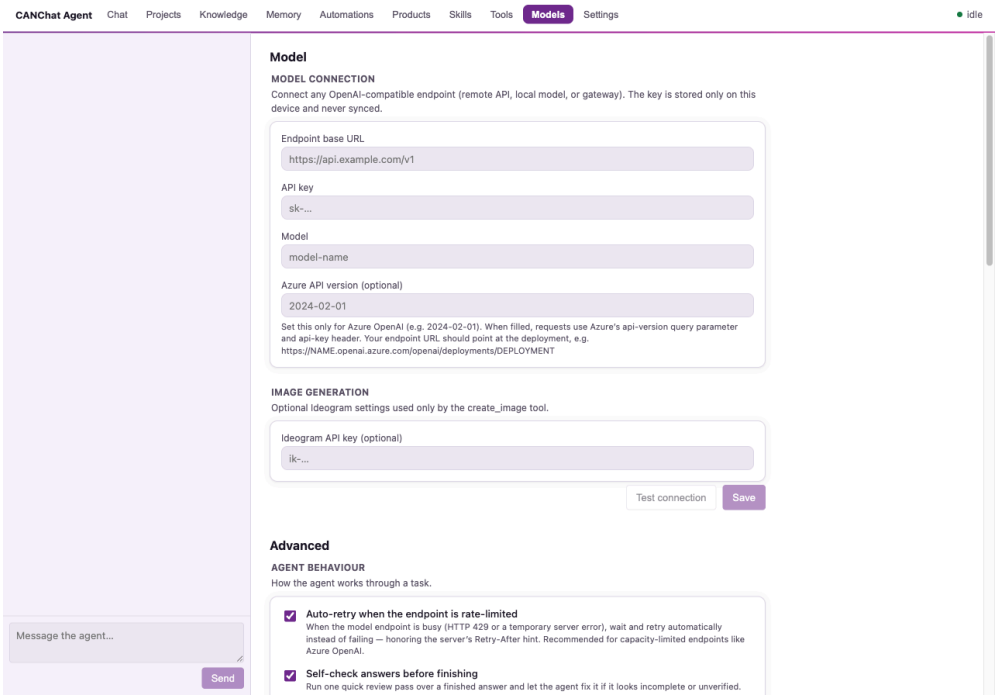
Settings — Model tab

Setting	Default	Description	Recommended
Endpoint base URL	*(empty)*	The OpenAI-compatible API	Required. Use your provider’s URL.

Setting	Default	Description	Recommended
		base, ending in `/v1` for most providers.	
API key	*(empty)*	Secret key, shown as dots; stored only on your device.	Required (unless a local model needs none — enter any placeholder it accepts).
Model	*(empty)*	Exact model name the endpoint expects.	Required. Pick a tool-calling model for automation.

(The interface **Language** picker now lives on the Workspace **Settings** page.)

Models page — Advanced section



Settings — Advanced tab

Setting	Default	Description	R
API version	*(empty)*	Set this **only for Azure OpenAI** (e.g. `2024-02-01`). Its presence switches the adapter to Azure mode (`api-key` header + `?api-version=`).	Leav stanc style
Auto-retry when rate-limited	**On**	When the endpoint is busy (HTTP **429** or a transient 5xx), automatically wait and retry instead of failing — honoring the server's `Retry-After` hint. A "retrying…" notice shows while it backs off.	Keep limit Azur off o

Setting	Default	Description	R
			failu imm
Temperature	*(unset)*	0–2 creativity dial; higher = more varied.	Leav the n 0–0. work
Max tokens	*(unset)*	Caps the length of each reply.	Leav you cost/
Embedder	On-device	Where knowledge-base text is turned into search vectors: **on-device** (transformers.js, nothing leaves your machine) or your **external `/embeddings` endpoint**. Switching requires re-indexing existing knowledge bases.	Leav unles requi embe
Hybrid search	**On**	Knowledge-base search blends meaning-based (semantic) matching with exact keyword matching, so identifiers/codes/names surface even when phrased differently.	Keep only sema
Embedding model	*(unset)*	Model for the `/embeddings` route used by knowledge bases (external embedder only).	Set i exter and need mod
Embedding endpoint URL / API key	*(use main)*	Optional separate host/key for embeddings.	Use onto servi
Transcription model	*(unset)*	Speech-to-text model for **voice prompts** (`/audio/transcriptions`).	Set style the n
Transcription endpoint URL / API key	*(use main)*	Optional separate host/key for transcription.	Use onto servi
SharePoint base URL	*(empty)*	Your SharePoint root (e.g. `https://contoso.sharepoint.com`) for searching SharePoint/OneDrive files.	Set i agen files signe
Azure app Client ID	*(empty)*	Enables mail search, calendar, drafting, and mailbox indexing via Microsoft Graph. Needs a one-time Azure AD app registration by your IT admin (see §5.7).	Requ mail/ featu for Shar

Setting	Default	Description	R
Azure tenant	*(empty → `organizations`)*	Graph OAuth tenant id.	Leav your requi tenar
Custom instructions	*(empty)*	Extra guidance appended to the agent's built-in instructions (tone, defaults, house style).	Opti short

Test connection and **Save** sit under the connection card; the Advanced section has its own **Save**. Each saves only its own fields, so saving one section never reverts the other. Settings take effect on Save.

Skills page

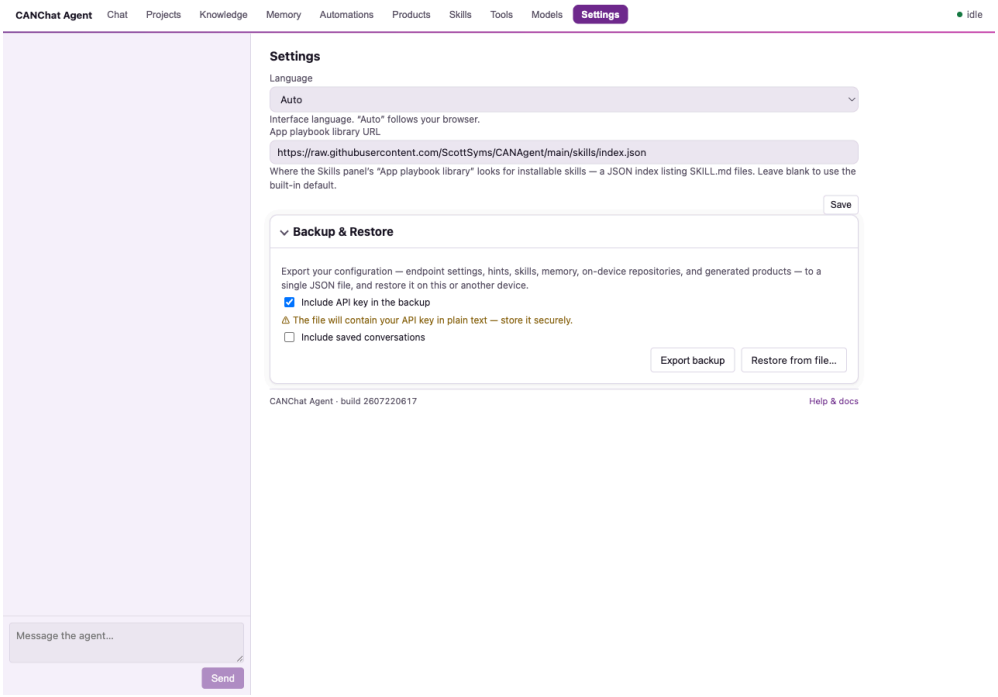
Manage reusable skills and app playbooks — see §5.6. The **App playbook library** here also polls a configurable **playbook index** (a hosted JSON list of installable `SKILL.md` files; defaults to the project's `skills/` folder) so listed skills install with one click.

Knowledge page

Your on-device page collections, with document/chunk counts and delete controls. Upload files or capture pages into a named base, then ask questions across them — see §5.7.

Tools, Memory, and Settings pages

The remaining device-data areas each have a Workspace page: **Tools** (known sites and MCP tool servers), **Memory**, and **Settings** (language, the App playbook library URL, and Backup & Restore):



Workspace — Settings page

- **Known sites** (Tools page) — a directory of sites worth checking first. Each entry has a name, a web address (optional), a description, an optional search shortcut (a URL with `{query}` in it), and optionally a **tool server (MCP endpoint)** with a token. The agent consults this before falling back to a web search.

- **Memory** (Memory page) — toggle **"Remember things about me (stored only on this device)."** When on, the agent builds a small personal knowledge graph. Two kinds of things get saved automatically after each turn: durable facts about you (work, projects, preferences), and — if you ask it to remember or discuss an article or page in depth — the named people, organizations, places, dates, and events it read there, linked by relationships (e.g. "Acme Corp —announced→ merger"). You can also say "remember that…" or "forget…" directly — that always works regardless of the slider below. Once saved, the agent answers directly from what it already knows instead of re-searching. A slider, **"Only auto-save facts the agent is at least this confident about,"** appears once Memory is on: raise it to make the automatic, opportunistic saving more conservative (0 = save anything the agent judges durable, the default; higher values require it to be surer). It only affects automatic saving — it never blocks something you explicitly ask it to remember. **Probe environment** (shown only when memory is on) fills memory with what the extension can detect about you — your Microsoft 365 name / work email / sign-in (AD) username from the signed-in session, the work systems you currently have open, and your locale/timezone. It runs entirely on-device and adds only new facts. Click **Manage memories** to open the full editor (in its own tab): search by text, filter by status or kind, edit a memory's text, **Confirm** it to clear a "stale" flag, or delete it — each entry shows the excerpt it was learned from, plus a link to the source article/page when the fact was extracted from one. Its **Relationships** list is clickable — click either end of a relationship to jump the detail view to that node, so you can walk the graph hop by hop (e.g. from an article's event to the organization it names, then to

another event that organization is linked to). A memory that hasn't been reconfirmed in a while is marked stale automatically (never deleted) so you know it might be out of date. Capacity is 500 entries; secrets are never saved.

The same toggle also enables **automatic lessons**: after a substantial or corrected task (one with a real plan, several tool calls, or a self-correction), the agent may quietly save one short behavioral tip for next time — e.g. "check the Outlook tool before driving the Outlook web UI." Lessons never contain your personal facts, page content, or secrets — only agent behavior — and are capped at 50, stored on-device, and included in Backup & Restore. There's no separate UI for them yet; turning Memory off stops new lessons from being saved and used.

- **Backup & Restore** (Settings page) — export everything to one JSON file and restore it later or on another machine. Options: **Include API key** (warned — plain text) and **Include conversations** (warned). Restore overwrites current settings, hints, skills, memory, and lessons, and replaces same-named knowledge bases.

The Workspace console

The Workspace is the full-tab console the **Settings** gear opens: the same conversation plus a dedicated page per management area — **Chat**, **Projects**, **Knowledge**, **Memory**, **Automations**, **Products**, **Skills**, **Tools**, **Models**, and **Settings** (language, playbook library, Backup & Restore) — alongside result viewers for data tables and images. **Models** carries the connection card, the full **Advanced** section, and **Model profiles & routing** (see next) for routing background work to a different model. Changes made in the workspace and the side panel share the same on-device storage, so either surface always reflects the latest state.

Model profiles and routing

By default every model call — the main chat, plus background work like generating a conversation title, learning from a task, or extracting a memory — uses the one model configured at the top of the Models page. If you want background work to run on something cheaper or fully local (e.g. Ollama) while keeping the main conversation on your strongest model, add a **profile** (a second named endpoint) on the Models page and assign it to one or more roles:

Role	Covers
Utility	Conversation titles/summaries, the self-check pass on a draft answer, skill creation, compacting old tool output, and improving knowledge-base search queries.
Reflection	Learning a behavioral lesson from a task, and extracting/merging durable memory facts.
Plan	The scoped research subtasks a multi-step plan spins up.

Role	Covers
Vision	Reading text out of a page screenshot when normal extraction fails (OCR).

A role with no profile assigned just uses the main model — nothing changes until you explicitly assign one. Tag a profile **Local** only if it's genuinely private (e.g. running on your own machine); the **"Restrict background tasks to local-tagged profiles"** toggle, when on, refuses to route any role to a profile that isn't tagged Local, falling back to the main model instead — so a mistaken cloud assignment can never send background work off-device.

Projects

Projects let you keep separate lines of work from mixing — client A's saved facts never leak into client B's answers, and vice versa. Create one from the **Projects** page in the Workspace, then switch between them from the small dropdown in the sidebar header (or the Projects page itself). Whichever project is active governs:

- **Conversations** — a new chat is tagged with the active project when you send its first message; the History list only shows conversations for the active project (plus any never assigned to one).
- **Memory** — facts you (or the agent) save while a project is active are tagged to it, and stay invisible while a different project is active.
- **Skills and known sites/capabilities** — anything you explicitly assign to a project (an optional field on the add/edit form, shown once you have at least one project) behaves the same way.

Nothing outside a project is ever hidden — records you never assign to a project stay visible everywhere, and switching projects only ever *filters* what's shown, it never deletes anything. Deleting a project does not delete its conversations, memory, skills, or capabilities; they simply become invisible until you either switch back or reassign them.

Automations

The **Automations** page in the Workspace is where background work — anything the agent does without you watching turn by turn — lives:

- **Scheduled tasks.** Ask the agent to "schedule a task that checks my calendar every morning at 8am" and it appears here: pause/resume it, delete it, and see its recent runs (what it found, or what went wrong) shown in full, not truncated — no need to hover to read it. When a run finishes you'll get a desktop notification (click it to jump straight to this page) even if the sidebar wasn't open; if that run created a file (e.g. "create a PowerPoint of today's headlines"), it's saved to the **Products** page (see below) rather than forced into an OS download prompt — handy if you have several jobs running and don't want a download popup for each one.
- **Workflows.** A named, ordered chain of your existing skills — e.g. a "Morning routine" workflow that runs `/research` then `/search-mail` in sequence. Create one from the page (name it, list the skills in order); it's not a new capability, just a saved shortcut through skills you already have.

- **Event triggers.** "When I open a page on this site, run this skill or workflow" — e.g. automatically triage a ticket queue whenever you open your Jira board. Set the site, what to run, and (optionally) a cooldown so it doesn't refire every time you're back on that tab within the same session.

Every one of these — scheduled, workflow, or triggered — runs under the exact same safety rule as anything else unattended: it can read, search, and gather freely, but a state-changing action (clicking something, filling a form, sending mail) still needs your approval, so it simply pauses and waits rather than acting without you. Nothing here can do something a normal chat message couldn't already do with your approval; these just decide *when* it runs. Running a SQL query against loaded data is also switched off unattended, even though it can't change anything outside the extension — it's off by default until there's someone present to sanity-check the result.

Products

Files a scheduled task or event trigger generates — a PowerPoint, a Word document — land on the **Products** page in the Workspace, not your OS Downloads folder. Each entry shows its filename, when it was created, its size, and which task or trigger produced it; click **Download** to save a copy wherever you like, or ✕ to delete it once you're done. They're stored on-device (in the browser's private file system) and never expire on their own, so a week of morning digests won't scatter files across your Downloads folder — they're all in one place, timestamped, until you clear them.

---

7. Common workflows

Each is a request you type into the composer; the agent handles the steps.

Goal	What to say	What happens
Summarize a webpage	"Summarize this page."	Reads the active tab, returns a structured summary.
Research a topic	"/research the impact of X" or "Research X and cite sources."	Searches, reads 2–3 sources, cross-checks, cites.
Analyze multiple tabs	"Compare what's across my open tabs."	Reads all tabs (with your approval), groups common vs. unique findings.
Search an authenticated site	"Find my open Jira tickets." / "Search SharePoint for the Q3 plan."	Uses your existing logged-in session; pauses for login if needed.
Collect data into a spreadsheet	"List every product on these pages with name and price as a table."	Gathers rows, emits a Download CSV/JSON card.

Goal	What to say	What happens
Write a document	“Draft a one-page summary as a Word document.”	Produces a downloadable `.docx`.
Save pages to ask later	“Save this page to a knowledge base called research.” then “#research what did they say about pricing?”	Stores on-device; answers with citations.
Use a tab group as a source	After it opens several tabs: “Summarize the Wolf group.”	Reads every tab in that named group.
Combine data with documents	“Which projects went over budget, and what do their status reports say?”	Queries the loaded dataset for the exact answer, then searches saved pages for matching narrative, and cites both.
Run browser automation	“Open the compose window and start a draft to …”	Maps the page, then asks approval before each click/type/submit.
Teach a new app	“/learn” on a site you use often.	Explores the app and saves an app playbook for next time.
Connect an MCP service	Add the MCP server under Known sites, then “Use the tools to …”	Lists the server’s tools and calls them (with approval).

> 📷 **Screenshots to capture manually — credentialed integrations.** Two workflows depend on real accounts/servers and can’t be reproduced in the offline harness:
>
> - **SharePoint search results.** After setting a SharePoint base URL and asking “Search SharePoint for …”, the answer lists ranked results with snippets, source links, and author/modified details. *Why manual:* requires a real signed-in SharePoint/Microsoft 365 session. *Suggested file:* `docs/user-guide/screenshots/08-sharepoint-results.png`.
> - **Microsoft 365 unified search.** With a signed-in M365 session, “my last five emails from Brian Ray” or “the last Word file I edited on my SharePoint site” are answered by the `microsoft365_search` tool (mail via Outlook on the web, files via SharePoint/OneDrive) — no setup or token. *Why manual:* needs a live session; the mail side uses Outlook’s web endpoint and is best-effort, falling back to the `/search-mail` skill.
> - **A live MCP tool call.** After registering an MCP server under Known sites and asking the agent to use it, you’ll see the tool-discovery step, an **Approve action?** card for `call_mcp_tool`, and the result. *Why manual:* needs a real reachable MCP server and token. *Suggested file:* `docs/user-guide/screenshots/09-mcp-call.png`.

8. Privacy and security

What is stored, and where

Everything CANChat Agent keeps lives **on your device**, in the browser's local extension storage (and OPFS for knowledge bases). Nothing is sent to the project's authors or any third party beyond the model endpoint(s) *you* configure.

Data	Where it's stored
Endpoint URL, **API key**, model, all settings	`chrome.storage.local` (this device)
Skills, app playbooks, known sites (hints), memory, language	`chrome.storage.local`
Conversation history (and labels)	`chrome.storage.local`
Knowledge bases (page text + search vectors)	OPFS (browser's private on-device file storage)

What is transmitted, and to whom

- **Your messages and the page text the agent reads** are sent to the **model endpoint you configured** so it can answer. Choose an endpoint you trust; treat page content the agent reads as shared with that provider.
- **Embeddings:** to make knowledge bases searchable, saved page text is sent to your configured **embeddings endpoint**.
- **Voice:** recorded audio is sent to your configured **transcription endpoint**.
- **Web searches** go through your browser's **default search engine**, as if you searched yourself.
- **SharePoint / authenticated sites:** requests use your **existing browser session (cookies)** — the agent does not see or store your passwords.

API-key handling

The key is stored locally, shown as dots in the UI, and sent **only** to the endpoint(s) you set. Backups **exclude conversations by default** and let you **omit the API key**; if you include it, the file holds the key in plain text — store it securely.

Browser permissions and the approval model

- **State-changing actions** (click, type, submit, run JavaScript, read all tabs, call external/MCP/WebMCP tools, save a playbook) **require your approval** every time, with a plain-language reason.
- **Site access** beyond the active tab may prompt an **Allow this site / Allow all sites** card — you decide the scope.

- **Logins** are never automated; the agent pauses and you sign in yourself.

Security recommendations

1. Use an endpoint and provider you trust with page content.
2. Read the **reason** on each approval before allowing it; **Deny** anything unexpected.
3. Keep **Memory** off unless you want personalization, and review/clear it periodically.
4. When backing up, leave **Include API key** unchecked unless you need it, and store the file securely.
5. Prefer **Allow this site** over **Allow all sites** when granting access.

---

9. Accessibility review

This section reports findings from inspecting the implementation and is honest about gaps.

Keyboard navigation — good

- **Enter** sends; **Shift+Enter** inserts a newline.
- The `@`/`#`/`/` menus are fully keyboard-driven: ↑/↓ move, **Enter/Tab** accept, **Esc** dismisses.
- The header's **More** menu is a real ARIA menu: `aria-haspopup` trigger, `role="menu"`/`menuitem`, arrow-key navigation, and Escape returns focus to the trigger.
- Controls show a visible **focus ring** for keyboard users (`:focus-visible`), added in the recent visual pass.
- Collapsible workspace sections use native `<details>`, so they're keyboard-operable by default.

Screen-reader compatibility — mostly good

- The composer is a custom editor exposed as `role="textbox"` with `aria-multiline="true"`.
- Icon-only buttons carry `aria-label`/`title`; snapshot images have `alt` text.
- **Recommendation:** add an `aria-live` region so status changes (Thinking → Acting → answer) and new messages are announced automatically; today a screen-reader user may need to navigate to discover updates.

Contrast and color — good, with notes

- Colors were tuned to **WCAG AA** in the recent redesign (darkened secondary text; brand purple meets contrast for text and buttons).
- Status is conveyed by **text + a colored dot**; the page-context list also pairs each status dot with a text tag (e.g. `auth_required`), so meaning isn't color-only.

- **Recommendation:** verify the amber "stale"/"warn" tags meet AA on their tinted backgrounds at small sizes.

Focus indicators — good

Buttons, links, tabs, and inputs all show a focus outline or ring. Modal overlays (Settings, History) don't trap focus — see below.

Form accessibility — good

Every field uses a `<label>` wrapping its control (label-control proximity), so names are programmatically associated.

Motion — handled

The status-dot pulse and the mic-recording pulse respect `prefers-reduced-motion` (animation is disabled when the OS setting is on).

WCAG concerns / recommendations (summary)

1. **Live announcements** (`aria-live`) for status and incoming messages — *highest-value addition*.
2. **Focus management** in the Settings/History overlays: move focus into the dialog on open, restore it on close, and consider a focus trap + `Esc` to close (`role="dialog"`/`aria-modal`).
3. **Touch targets:** icon buttons are ~32–36 px; nudging toward 44 px would fully meet AAA target-size guidance.
4. Confirm small tinted tags (warn/stale/MCP) clear AA.

---

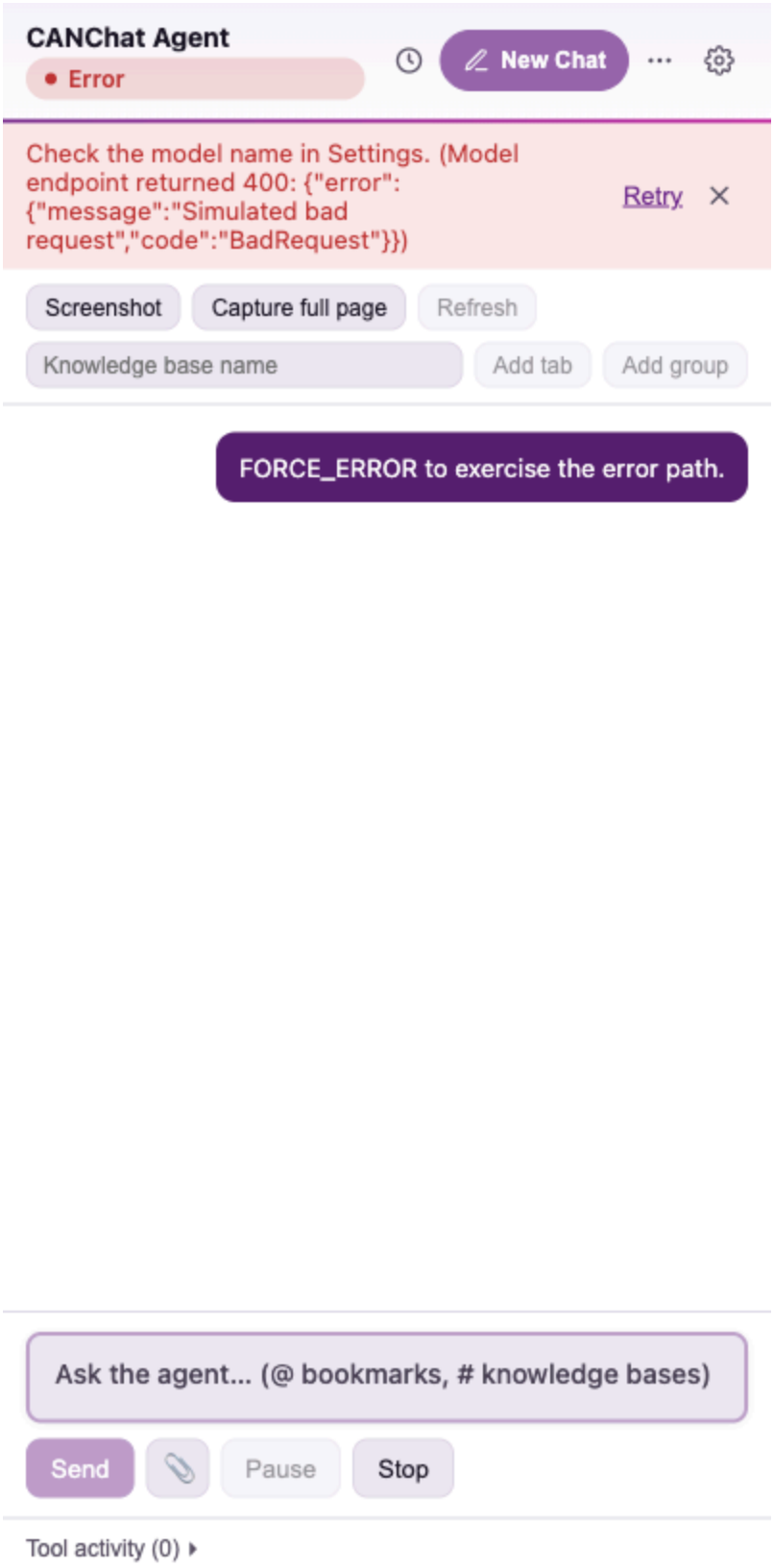
10. Troubleshooting

Symptom	Likely cause	Resolution
"No model configured" banner; agent won't run	No endpoint/key/model saved	Open Workspace → Models, fill all three, **Save**.
Test connection fails: "check your API key"	Wrong/expired key, or wrong auth style	Re-enter the key. For **Azure**, set **API version** (Advanced) so it uses the `api-key` header.
"Could not reach the model endpoint"	Wrong URL, server down, or CORS blocking	Verify the base URL (include `/v1` for OpenAI-style). If the endpoint blocks cross-origin requests, **re-save settings**

Symptom	Likely cause	Resolution
		to grant access; approve any **Allow site** prompt.
"returned 400 … model"	Model name not recognized by the endpoint	Correct the model name to one the endpoint lists.
"429 / Too Many Requests" / "rate-limited"	The endpoint (often Azure OpenAI) is over capacity	With **Auto-retry** on (default) the agent backs off and retries automatically — you'll see a "retrying…" notice. If it still fails, the endpoint is saturated; wait and try again, or raise your quota.
Agent answers but won't click/automate	Model lacks tool-calling	Switch to a model that supports tools/function calling.
"Approve action?" keeps appearing	Working as designed	Each state-changing step asks once; **Approve** to proceed or **Deny** to stop.
Task paused, asks me to log in	Page hit a sign-in wall	Sign in to the site in the browser, then **Resume**.
Permission card: "Allow this site"	Agent needs access to that site	Choose **Allow this site** (preferred) or **Allow all sites**.
Microphone button missing	No transcription model set	Add one under Workspace → Models → Advanced.
Mic does nothing / permission error	Side panel can't prompt for the mic	Use the tab it opens to allow the microphone once, then tap the mic again.
"Screenshot" does nothing	Restricted page (`chrome://`, Web Store, PDF viewer)	These can't be captured; ask about the page's text instead.
PDF/Office file "no text"	Scanned/image-only PDF, or legacy `.doc/.xls/.ppt`	Use a text-based PDF or a modern `.docx/.pptx/.xlsx`.
Knowledge-base search fails	Embeddings route/model not available	Set an **Embedding model** (and endpoint if separate) in Advanced; confirm the endpoint exposes `/embeddings`.
`run_javascript` blocked	The site's security policy (CSP) forbids it	The agent falls back to clicking via the element map; some pages remain limited.

Symptom	Likely cause	Resolution
Long task seems to stall	MV3 background worker idle, or step budget reached	The panel keeps it alive automatically; if it stops after many steps it hit the safety cap (40) — ask it to continue or narrow the task.
Everything looks wrong after an update	Stale build loaded	On `chrome://extensions`, click **Reload** on the extension.

General error recovery: when something fails mid-task, the panel shows a plain-language banner (check key / endpoint / model) plus the raw detail and a **Retry** that re-sends your last message:



Error banner with Retry

11. FAQ

Do I need an OpenAI account? No — any OpenAI-*compatible* endpoint works, including local models (Ollama, LM Studio) and corporate gateways. You just need one endpoint URL, key, and model name.

Does it cost money? The extension is free, but **you** pay your model provider for usage. Local models cost nothing beyond your hardware.

Is my data sent anywhere? Your messages and the pages the agent reads go to *your* configured endpoint(s). Settings, history, memory, and knowledge bases stay on your device. See Privacy.

Can it act without asking? No page-changing action runs without your approval. Reading the page you're on is automatic; changing it is gated.

Will it log into sites for me? No. It uses sessions you're *already* logged in to and pauses for you to sign in when needed.

What's the difference between a Skill and an App playbook? A skill is a reusable instruction you invoke with `/name`. An app playbook is a skill bound to a website that loads automatically when you're on that site.

What are MCP and WebMCP? MCP connects the agent to an external tool server you register under Known sites. WebMCP lets the agent use tools a web page exposes itself. Both external/page tool calls require your approval.

Where did my conversation go when I clicked the compose icon? Nowhere — "New chat" keeps the old conversation in **History**.

Can I move my setup to another computer? Yes — **Workspace → Settings → Backup & Restore** exports one JSON file you can restore elsewhere.

Which languages are supported? The interface is English and French; answers follow whatever language your model produces.

Does it work in Firefox/Safari? No — it requires a Chromium browser (Chrome or Edge) version 116+.

---

12. Quick reference (cheat sheet)

Most common actions

Do this	Where
Open the assistant	Toolbar icon → side panel
First-time setup	Welcome card → endpoint, key, model → Test → Save & start

Do this	Where
Ask about the current page	Type a question → **Enter**
Research / multi-tab	`/research …` or "compare my open tabs"
Insert a bookmark / knowledge base / skill	`@` / `#` / `/` in the composer
Save a page to ask later	Toolbar → name → **Add tab** / **Add group**
Start fresh (keep history)	Header **New Chat** button
Save conversation to a file	Header ⋯ **More** → Save conversation
Find a past conversation	Header **clock** (History) → search / sort / labels
Change text size / language	⋯ **More** → `A−` `100%` `A+` / Workspace → Settings

Keyboard

Key	Action
Enter	Send message
Shift+Enter	New line in the composer
↑ / ↓	Move within the `@/#//` menu
Enter / Tab	Accept the highlighted menu item
Esc	Dismiss the menu

Settings locations (all in the Workspace — the header gear opens it)

Setting	Workspace page
Endpoint, key, model, Azure version, image key	**Models**
Behaviour, temperature, max tokens, custom instructions, embeddings, transcription, SharePoint, Graph	**Models → Advanced**
Skills & app playbooks	**Skills**
Known sites & tool servers	**Tools**
Memory toggle & editor	**Memory**
Knowledge bases	**Knowledge**
Language, playbook library URL, backup/restore	**Settings**

Run controls: Send · Pause/Resume · Stop. **Safety:** Approve/Deny on each state-changing step.

13. Known limitations

- **No bundled model.** You must supply an OpenAI-compatible endpoint, key, and model; nothing works until configured.
- **Model-dependent features.** Browser automation needs a **tool-calling** model; reading snapshots / full-page captures needs a **vision** model.
- **Voice setup friction.** Voice needs a transcription endpoint *and* a one-time microphone grant from a separate tab (the side panel can't prompt for it).
- **Page-reading limits.** Scanned image-only PDFs and legacy `.doc/.xls/.ppt` yield no text; some apps require snapshot+vision; `run_javascript` can be blocked by a site's content-security policy.
- **Restricted pages.** `chrome://` pages, the Chrome Web Store, and similar can't be scripted or captured.
- **Step budget.** Tasks are bounded (soft 20 steps, extendable to a hard cap of 40. to control cost; very long jobs may need to be split.
- **Single shared conversation.** One agent/conversation is shared across panels and windows; opening the panel elsewhere shows the same session.
- **Ephemeral background worker (MV3).** The service worker can be evicted when idle; the panel sends keepalives during tasks, but extremely long idle gaps may reset background state (your data is safe in storage).
- **Retrieval quality costs LLM calls.** Knowledge-base search sends the stored chunk text (not just the query) to your **LLM endpoint** for reranking, and the query itself for paraphrase generation — this happens even with the on-device embedder, which only keeps *embedding* local, not the reranking pass. Turn off **Hybrid search** in Workspace → Models → Advanced only removes the keyword-fusion step, not the rerank/paraphrase calls.
- **Languages.** Interface localization is English and French only.

Future enhancement opportunities

- `aria-live` announcements and dialog focus management (see Accessibility).
- Per-conversation (not global) agent sessions.
- OCR for scanned PDFs.

Appendix A: Complete feature inventory

Derived directly from source; "✅ verified" means traced to its implementation.

Shell & chat - Side-panel UI, opens from toolbar icon ✅ (`serviceWorker.ts`, `manifest.json`) - English/French in-app localization ✅ (`i18n.tsx`) - Text-size scaler `A−` `100%` `A+` in the header More menu ✅ (`Sidebar.tsx`, `OverflowMenu.tsx`) - Status pill: idle/thinking/acting/paused/awaiting_approval/auth_required/error with reduced-motion-aware pulse ✅ (`types.ts`, `styles.css`) - Composer with Enter-to-send, `@` bookmark / `#` knowledge-base / `/` skill menus ✅ (`ChatPanel.tsx`) - Markdown answers

with Copy, source-citation block, data-export and Word download cards ✅ (`ChatPanel.tsx`, `Markdown.tsx`) - New chat (keeps history), save conversation as HTML ✅ (`Sidebar.tsx`, `conversationExport.ts`)

Agent core - Plan panel (`set_plan`/`update_plan`), findings (`record_finding`), step budget (soft 20 → +10 → hard 40), context compaction at 90k chars ✅ (`agentRuntime.ts`) - Tool-activity log ✅ (`ToolActivityPanel.tsx`) - Approval gate for state-changing tools; auth-pause; site-permission prompt ✅ - Distill offer after substantial tasks (plan ≥3 or ≥4 tool calls) ✅ - **Scoped subtask delegation** (`run_subtasks`) — up to 12 isolated mini-loops (own context, tight tool allowlist, 3 at a time) for multi-page comparison/ research; only each subtask's compact conclusion returns to the main conversation ✅ (`agentRuntime.ts`) - **Automatic lessons** — after a substantial or self-corrected task, the agent may save one reusable behavioral tip (opt-in, shares the Memory toggle), merged with similar existing lessons and surfaced on matching future tasks ✅ (`agentRuntime.ts`, `loopHelpers.ts`)

Browser tools — full list in Appendix B.

Knowledge / integrations - On-device knowledge bases (OPFS) with embedding search, **on-device by default** (external `/embeddings` optional) ✅ (`repoIngest.ts`, `offscreen/repoStore.ts`, `localEmbed.ts`) - **Local folder indexing** via drag-and-drop, with incremental re-sync ✅ (`folderIndex.ts`, `RepositoriesSection.tsx`) - **Office 365 mailbox indexing, mail search, calendar, and drafting** via Microsoft Graph (OAuth/PKCE; needs a one-time Azure app registration), with optional hourly mailbox auto-refresh ✅ (`mailIngest.ts`, `graphAuth.ts`, `graphClient.ts`, `MailboxSection.tsx`) - **Hybrid search** (semantic + BM25 keyword fusion), multi-query paraphrase retrieval, and LLM reranking on knowledge-base search ✅ (`hybridSearch.ts`, `keywordSearch.ts`, `agentRuntime.ts`) - Capability Registry (bookmarks, MCP servers, known sites) incl. MCP servers ✅ (`CapabilitiesSection.tsx`, `mcpClient.ts`) - WebMCP bridge (page-exposed tools) ✅ (`webmcpBridge.ts`) - SharePoint search via signed-in session ✅ (`schemas.ts`, runtime) - Skills, app playbooks, `/learn`, curated playbook library (Outlook OWA/Live, Gmail, MarineTraffic, Jira Cloud) ✅ (`SkillsSection.tsx`, `curatedPlaybooks.ts`) - Durable graph memory (opt-in, ≤500 nodes): post-conversation reflection, embedding-indexed retrieval, staleness decay, and a full management page; and automatic lessons (opt-in, ≤50 entries) ✅ (`memoryGraph.ts`, `memoryIndex.ts`, `MemoryPage.tsx`, `MemorySection.tsx`, `storage.ts`)

Input/output - Voice prompts via `/audio/transcriptions` ✅ (`ChatPanel.tsx`, `llmProvider.ts`) - Page snapshots (downscaled JPEG) + full-page capture ✅ (`TabContextPanel.tsx`) - PDF, Office, and video-caption reading ✅ (`schemas.ts`,

`offscreen/docGen.ts`) - CSV/JSON export, `.docx` generation ✅ (`ChatPanel.tsx`, `docGen.ts`)

History & data - Auto-saved conversations; search, sort, colored labels & filter ✅ (`ConversationsScreen.tsx`, `LabelPicker.tsx`) - Conversation import/export; Clear all ✅ - Backup/Restore of settings, hints, skills, memory, language, knowledge bases; optional key/conversations ✅ (`BackupRestoreSection.tsx`)

Providers - Any OpenAI-compatible endpoint; standard (Bearer) and Azure (`api-key` + `api-version`) modes; separate endpoints/keys for chat, embeddings, transcription ✅ (`llmProvider.ts`)

Experimental / limited / undocumented behaviors

- **`read_app_content`** — best-effort reading of canvas-rendered apps (Google Docs/Sheets); explicitly a fallback that may return nothing.
- **`capture_full_page`** — vision-only, token-heavy "last resort."
- **`run_javascript`** — powerful but **CSP-blockable**; gated by approval.
- **Coordinate gestures** (`click_at`, `drag`, `scroll_wheel`) — for canvas/maps; rely on element rects and may be imprecise.
- **Curated playbook for `atlassian.net`** is matched by host; Jira Server (self-hosted) isn't covered by the curated entry.

Appendix B: Complete tool catalogue

Every action the agent can take (`src/shared/schemas.ts`). 🔒 **= requires your approval each time.** Memory tools appear only when Memory is enabled.

Reading tabs & pages: `list_tabs`, `get_active_tab`, `get_tab_content`, `read_app_content`, `capture_full_page`, `get_all_tab_contents` 🔒, `read_tab_group`, `read_pdf`, `read_office_document`, `get_video_transcript`.

Navigation & search: `navigate`, `open_url`, `search_web`, `search_known_sites`, `wait_for_page_state`, `wait_for_element`, `detect_auth_state`.

Page interaction: `get_element_map`, `click_element` 🔒, `fill_input` 🔒, `submit_form` 🔒, `press_keys` 🔒, `click_at` 🔒, `drag` 🔒, `scroll_wheel`, `query_pointer_target`, `run_javascript` 🔒.

Knowledge bases (RAG): `add_to_repo`, `search_repo`, `list_repos`, `run_subtasks` (isolated mini-loops for multi-page research).

External & in-page tools: `sharepoint_search`, `list_mcp_tools`, `call_mcp_tool` 🔒 , `list_webmcp_tools`, `call_webmcp_tool` 🔒 .

Producing output: `export_data` (CSV/JSON), `create_word_document` (.docx).

Agent meta: `set_plan`, `update_plan`, `record_finding`, `use_skill`, `save_app_playbook` 🔒 , `save_as_skill` 🔒 .

Memory (opt-in only): `save_memory`, `update_memory`, `delete_memory`.

Appendix C: Implementation vs. documentation discrepancies

Per the task's rule — *trust the implementation* — these are differences found between the code and existing docs/labels:

1. **Header icon (README is stale).** `README.md` describes the header as "CANChat Agent · status pill · 🗑 · ⚙" with a **trash can**. The implementation now uses a **compose icon labelled "New chat"** (the action keeps the previous conversation in History rather than deleting it — `agentRuntime.clearConversation` comments "Clear = new chat, not delete"). This manual documents the current behavior, and the README header line has since been corrected to match.
2. **Product name vs. package/repo name.** The user-facing product is **CANChat Agent** (manifest `name`, all UI), but the npm package is `browser-agent-extension` and the GitHub repo is `CANAgent`. Backups also accept the legacy tag `CANAgent`. No user impact; noted for clarity.
3. **"Hints" → "Known sites".** Older copy and the section's body called the known-sites directory **"Hints"** and referenced an "MCP server"; the current UI labels it **"Known sites"** and uses plainer wording ("tool server (MCP endpoint)"). This manual uses the current labels.
4. **Onboarding vs. "drops you into Settings".** The implementation shows a minimal three-field **welcome** on first run (not the full Settings modal); any documentation implying the latter is out of date.

No functional contradictions (a documented feature that doesn't work) were found: the existing README tracks the implementation closely apart from the cosmetic header note above.

End of manual. Screenshots regenerate via `npx playwright test walkthrough manual`; the document is verified against the source at version 0.1.0.